UNIVERSITY OF GHANA

COLLEGE OF BASIC AND APPLIED SCIENCES

DEPARTMENT OF COMPUTER SCIENCE

DCIT 204/308

DSA DOCUMENTATION PROJECT REPORT

Joint Semester Project

Project Title:	**UG Campus Smart Service Operations Optimizer**
Team Name:	**Struct-Enqueue 2**
Prepared By:	Documentation Team
Version:	1.1
Submission Date:	28th August 2026
Academic Year:	2026/2027

Revision History

This table records revisions made to this document throughout the project lifecycle.

Version	Date	Updated By	Description of Changes	Approved By
1.0	4th August 2026	Documentation Team	Initial documentation template created	Team Leader
1.1	27th August 2026	Documentation Team	Completed technical report, trace tables, proof sketches, counterexamples, performance analysis, and contribution records.	Team Leader

Table of Contents

Contents

PART I

TECHNICAL REPORT

1. Problem Statement, Assumptions, Input-Output Definitions and System Boundaries

Problem Statement

The University of Ghana campus comprises several academic, residential, administrative, health, and commercial facilities that may require different maintenance and operational services. Managing service requests across these locations involves prioritising requests, assigning available resources, and determining efficient routes between locations.

The **UG Campus Smart Service Operations Optimizer** applies Data Structures and Algorithms to support these activities. The system represents campus locations and routes as a graph and uses custom data structures and algorithms for organising service requests, searching and sorting data, graph traversal, shortest-path computation, resource assignment, and optimisation.

The project provides a practical application of Data Structures and Algorithms within a University of Ghana campus-service environment.

Assumptions

The system operates under the following assumptions:

- Each campus location, route, resource, and service request has a unique identifier.
- Routes connect recognised campus locations and contain valid distance, travel-time, and traffic information.
- Route weights used by graph algorithms are non-negative.
- Service requests contain valid category, urgency, location, deadline, and status information.

- Resources have known types, home locations, capacities, and availability statuses.
- The provided CSV datasets contain valid initial project data.
- PostgreSQL is used for persistent storage of project information.
- Route and traffic information represents predefined project data rather than real-time measurements.

Input Definitions

The principal inputs to the system are:

- **Locations:** location ID, name, area, type, latitude, and longitude.
- **Routes:** route ID, source location, destination location, distance, average travel time, and traffic factor.
- **Resources:** resource ID, resource type, home location, capacity, and availability status.
- **Service Requests:** request ID, source and destination locations, category, urgency, submission time, deadline, status, and assigned resource where applicable.

Output Definitions

The main outputs of the system include:

- searched and sorted records;
- prioritised service requests;
- BFS and DFS graph traversal results;
- shortest paths produced by Dijkstra's Algorithm;
- Minimum Spanning Trees produced by Prim's and Kruskal's Algorithms;

- resource-assignment results produced through greedy optimisation;
- optimised selections produced through Dynamic Programming; and
- correctness and performance results from algorithm testing.

System Boundaries

The project focuses on the **data-structural, algorithmic, database, and computational aspects of campus service optimisation**. Its scope includes custom data structures, searching and sorting, graph algorithms, request prioritisation, resource allocation, optimisation, PostgreSQL data storage, and algorithm testing.

Real-time GPS tracking, live traffic acquisition, web and mobile applications, and integration with external University information systems are outside the scope of the project.

2. Dataset Description, Data Dictionary and Database Schema

2.1 Dataset Description

The project uses four CSV datasets containing information required to model and optimise campus service operations. These datasets are loaded into a PostgreSQL database for persistent storage and algorithmic processing.

Dataset	Records	Description
locations.csv	60	Contains University of Ghana

		campus locations and their geographical details.
routes.csv	120	Represents connections between campus locations, including distance, travel time, and traffic factor.
resources.csv	35	Contains available service personnel and operational resources.
requests.csv	310	Contains maintenance and

		operational service requests submitted from campus locations.

The datasets collectively provide the information required for graph construction, searching, sorting, request prioritisation, resource assignment, routing, and optimisation.

2.2 Data Dictionary

Data Entity	Main Attributes	Purpose
Location	location_id, name, area, type, latitude, longitude	Identifies and describes campus locations used as vertices in the route network.

Route	route_id, from_location_id, to_location_id, distance_m, avg_time_min, traffic_factor	Represents weighted connections between campus locations.
Resource	resource_id, type, home_location_id, capacity, availability_status	Represents personnel or operational resources available for service requests.
Service Request	request_id, source_location_id, destination_location_id, category, urgency_level, time_submitted, deadline, status, assigned_resource_id	Stores service requests and the information required for prioritisation, scheduling, and resource assignment.

The service-request urgency level is restricted to values from **1 to 5**, while unassigned resources and optional destination information may be represented as null values in the database.

2.3 Database Schema

The PostgreSQL database consists of six main tables:

Table	Purpose
Locations	Stores campus location information.
Routes	Stores route connections between locations.
Resources	Stores available service resources.
service_requests	Stores maintenance and

	operational requests.
algorithm_runs	Stores algorithm performance results such as input size, execution time, and memory usage.
audit_events	Stores audit information relating to system operations and events.

The database uses **primary keys** to uniquely identify records and **foreign keys** to maintain relationships between entities. routes references locations; resources references their home locations; and service_requests references locations and assigned resources.

Indexes are also defined on frequently accessed attributes such as request status, urgency level, request location, route endpoints, resource availability, and algorithm name to improve database query efficiency.

3. System Architecture and Module Design

3.1 System Architecture

The **UG Campus Smart Service Operations Optimizer** follows a modular architecture in which different components are responsible for data storage, data representation, algorithm processing, service operations, and user interaction. This design promotes separation of responsibilities and allows individual Data Structures and Algorithms to be developed, tested, and integrated independently.

The overall system flow is:

CSV Datasets → PostgreSQL Database → Models and Data Structures → Algorithms and Graph Processing → Service Layer → Console Interface

3.2 Module Design

Module	Main Components	Responsibility
Database Module	DatabaseConnector, SchemaSetup, CSVLoader	Establishes the PostgreSQL connection, creates the database schema, and loads project datasets.

Model Module	Location, Route, Resource, ServiceRequest, AlgorithmRun, AuditEvent	Represents the main entities and records processed by the system.
Data Structure Module	Dynamic Array, Linked List, Stack, Queue, Circular Queue, Deque, Priority Queue, BST, Red-Black Tree, B-Tree, Hash Table, Set, Map, Disjoint Set	Provides custom structures for storing, indexing, prioritising, organising, and processing campus-service data.
Algorithm Module	Linear Search, Binary Search, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Greedy, Dynamic Programming	Performs searching, sorting, resource allocation, and optimisation.

Graph Module	Graph, BFS, DFS, Dijkstra, Prim, Kruskal, GraphDataLoader	Represents the campus route network and performs traversal, shortest-path, and Minimum Spanning Tree operations.
Service Module	RequestDispatcher, SchedulingEngine, IndexingEngine	Coordinates request dispatching, scheduling, prioritisation, and indexed searching within the application.
User Interface Module	ConsoleMenu	Provides the console-based interface through which users access the system's operations.

Application Entry Point	Main	Initialises the application and coordinates database, graph, service, and console components.

The modular architecture enables the database, custom data structures, algorithms, graph-processing components, service logic, and console interface to operate together while maintaining clear separation between system responsibilities.

4. Data Structure Implementation

The project implements custom linear and non-linear data structures to support the storage, organisation, searching, prioritisation, routing, and optimisation of campus service data.

Data Structure	Implementation	Role in the Project
Dynamic Array	Array-backed structure with automatic capacity expansion.	Provides flexible sequential storage for project records.
Doubly Linked List	Node-based structure containing previous and next references.	Supports efficient insertion, deletion, and sequential traversal.
Stack	Last-In-First-Out (LIFO) structure.	Supports operations requiring reverse-order processing.
Queue	First-In-First-Out (FIFO) structure.	Supports sequential processing of requests and

		graph traversal.
Circular Queue	Array-based queue with circular front and rear movement.	Reuses storage efficiently for cyclic processing.
Deque	Double-ended queue supporting operations at both ends.	Allows flexible processing of normal and urgent requests.
Priority Queue	Heap-based priority structure.	Supports priority-based request processing and minimum-cost selection in graph algorithms.
Hash Table	Stores key-value pairs using hashing and	Provides efficient indexing and

	collision handling.	lookup of system records.
Map	Key-value structure built using the custom hashing implementation.	Supports association and retrieval of records using unique keys.
Set	Stores unique elements and prevents duplicate membership.	Supports membership and uniqueness operations within the system.
Binary Search Tree (BST)	Organises key-value records according to binary-search ordering.	Supports ordered searching, insertion, deletion, and tree traversal.

Red-Black Tree	Self-balancing Binary Search Tree using node colouring and balancing operations.	Maintains efficient searching and updates by preventing excessive tree imbalance.
B-Tree	Multi-way balanced tree capable of storing multiple keys per node.	Supports efficient indexing and represents structures commonly used for database-oriented searching.
Disjoint Set (Union-Find)	Uses parent relationships with efficient union and find operations.	Supports connectivity checking and cycle detection in Kruskal's Algorithm.

Graph	Represents campus locations as vertices and routes as weighted edges.	Provides the network representation required by BFS, DFS, Dijkstra, Prim, and Kruskal algorithms.

The linear structures support general storage and request processing, while the hashing and tree structures provide efficient searching and indexing. The **Priority Queue** and **Disjoint Set** directly support graph algorithms, and the **Graph** models the University of Ghana campus route network for traversal, shortest-path, and Minimum Spanning Tree operations.

The use of different data structures enables the project to compare alternative approaches to storage, retrieval, ordering, prioritisation, and connectivity while demonstrating their practical application to campus service operations.

5. Algorithm Implementation

The system implements searching, sorting, graph, greedy, and dynamic programming algorithms to support data retrieval, request processing, campus-network analysis, resource allocation, and optimisation.

Algorithm	**Implementation and Application**
Linear Search	Examines elements sequentially until the target is found or the dataset is exhausted. It

	supports searching where data is not necessarily sorted.
Binary Search	Repeatedly divides a sorted search range into halves until the target is found or the search interval becomes empty. It provides efficient searching of sorted records.
Selection Sort	Repeatedly identifies the smallest remaining element and places it in its correct position. It provides a

	simple comparison-based sorting approach.
Insertion Sort	Builds a sorted portion of the array by inserting each new element into its correct position. It performs efficiently on small or nearly sorted datasets.
Merge Sort	Recursively divides the dataset into smaller subarrays and merges them in sorted order. It provides efficient and predictable

	sorting performance.
Quick Sort	Partitions the dataset around a pivot and recursively sorts the resulting partitions. It provides efficient average-case sorting performance.
Breadth-First Search (BFS)	Traverses the campus graph level by level using a queue and is used for reachability and network exploration.
Depth-First Search (DFS)	Explores the campus graph along a path before

	backtracking and supports connectivity and reachability analysis.
Dijkstra's Algorithm	Determines the shortest path between campus locations in the weighted route network using distance relaxation and priority-based selection.
Prim's Algorithm	Constructs a Minimum Spanning Tree by repeatedly selecting the minimum-weight edge that connects a new

	vertex to the growing tree.
Kruskal's Algorithm	Constructs a Minimum Spanning Tree by processing edges in increasing order of weight and using Disjoint Set operations to prevent cycles.
Greedy Algorithm	Assigns service requests to the nearest available resources according to the current best local choice.
Dynamic Programming	Applies the 0/1 Knapsack approach to select an optimal

	combination of requests or route options within a specified time or budget constraint.

The searching and sorting algorithms support efficient retrieval and ordering of system records. BFS, DFS, Dijkstra, Prim, and Kruskal operate on the campus graph to support traversal, routing, and network optimisation. Greedy allocation provides rapid resource assignment, while Dynamic Programming is used where an optimal selection must be obtained under defined constraints.

Together, these algorithms demonstrate multiple algorithmic strategies, including sequential search, divide-and-conquer, comparison-based sorting, graph traversal, shortest-path computation, Minimum Spanning Tree construction, greedy selection, and optimisation through tabulation.

6. Correctness Evidence

The correctness of the implemented data structures and algorithms is evaluated using **JUnit automated tests**, known expected outputs, boundary cases, invalid inputs, and, where applicable, real records from the PostgreSQL campus database.

Component	Correctness Evidence
Linear Search	Tests verify successful searches,

	unsuccessful searches, and rejection of invalid/null input.
Binary Search	Tests verify successful and unsuccessful searches on sorted data and demonstrate the effect of violating the sorted-input precondition.
Sorting Algorithms	Selection Sort, Insertion Sort, Merge Sort, and Quick Sort are tested on unsorted, empty, single-element, already sorted, reverse-sorted, and duplicate-

	containing arrays.
Linear Data Structures	Tests verify insertion, removal, ordering behaviour, capacity handling, empty/full conditions, and boundary cases for structures such as Dynamic Array, Queue, Circular Queue, Deque, Stack, and Priority Queue.
Trees and Indexing Structures	BST, B-Tree, Red-Black Tree, Hash Table, Set, and Map implementations are tested for

	their principal insertion, searching, retrieval, uniqueness, and structural operations.
Disjoint Set	Tests verify union, find, connectivity, path compression, and union-by-rank operations used for cycle detection.
BFS and DFS	Known graphs are used to confirm expected traversal behaviour and correct handling of isolated vertices.

Dijkstra's Algorithm	A known weighted graph verifies that the minimum-cost path and predecessor relationships are correctly determined, including handling of unreachable vertices.
Prim's Algorithm	A known graph is verified to produce a Minimum Spanning Tree with the expected number of edges and total cost.
Kruskal's Algorithm	MST size and total cost are verified and

	compared with Prim's result to provide additional correctness evidence.
Greedy Algorithm	Tests use PostgreSQL campus data to verify that resources are assigned according to the nearest available choice and are not incorrectly reused within an allocation process.
Dynamic Programming	Results are cross-checked against a brute-force solution for the same request data.

	Tests also verify budget constraints, exact-fit cases, zero-budget behaviour, and invalid inputs.

The testing approach combines **unit testing, expected-result comparison, boundary testing, precondition testing, and cross-validation between algorithms**. This provides evidence that the individual components behave according to their defined requirements under representative normal and exceptional conditions.

7. Performance Analysis

The performance of the implemented algorithms was evaluated using both **theoretical complexity analysis** and **empirical execution-time and memory measurements**. Let **n** represent input size, **V** the number of graph vertices, **E** the number of graph edges, **R** the number of service requests, **M** the number of resources, and **B** the optimisation budget.

Algorithm	**Expected Time Complexity**	**Expected Space Complexity**
Linear Search	O(n)	O(1)
Binary Search	O(log n)	O(1)
Selection Sort	$O(n^2)$	O(1)
Insertion Sort	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n)
Quick Sort	O(n log n) average	O(log n) average

BFS	O(V + E)	O(V)
DFS	O(V + E)	O(V)
Dijkstra	O((V + E) log V)	O(V + E)
Prim	O(E log V)	O(V + E)
Kruskal	O(E log E)	O(V + E)
Greedy Resource Assignment	O(R × M)	O(R + M)
Dynamic Programming	O(nB)	O(nB)

Experimental results generally reflected the expected behaviour of the algorithms. At an input size of **10,000**, Binary Search completed in approximately **0.000088 ms**, while Linear Search required approximately **0.007002 ms**, demonstrating the advantage of logarithmic searching on sorted data.

For sorting at **n = 10,000**, Selection Sort required approximately **186.9602 ms**, Insertion Sort **97.7369 ms**, Merge Sort **5.7776 ms**, and Quick Sort **1.1268 ms**. These results show that Merge Sort and Quick Sort scaled more efficiently than the quadratic sorting algorithms for larger inputs.

For graph operations at **1,000 vertices**, BFS and DFS remained relatively fast, while Dijkstra, Prim, and Kruskal required additional processing for weighted-path and Minimum Spanning Tree calculations.

Dynamic Programming showed increasing memory consumption as the input size increased, reaching approximately **8096.570 KB** at **n = 10,000**, which is consistent with the use of a two-dimensional tabulation structure.

Overall, the experiments support the theoretical expectations of the implemented algorithms, although individual execution times may vary because of JVM optimisation, hardware conditions, and system activity during testing.

8. Database Integration Evidence

The project integrates with a **PostgreSQL database** through Java Database Connectivity (JDBC). The DatabaseConnector class manages the connection and provides methods for executing database queries and updates. Database credentials are read from the project .env file, system environment variables, or fallback defaults.

The SchemaSetup component creates the required database tables and indexes using CREATE TABLE IF NOT EXISTS, while the CSVLoader populates the database from the project datasets. Data is loaded in the order **locations → routes → resources → service requests** because later records depend on earlier records through foreign-key relationships.

The CSV loader uses prepared SQL statements to insert location, route, resource, and service-request records into their corresponding tables. It also handles optional service-request values, such as unassigned resources, by storing them as NULL where appropriate.

Database information is further integrated with the algorithmic components through GraphDataLoader. Campus locations are retrieved from the database and converted into graph vertices, while route records are converted into weighted graph edges. The effective route weight is calculated using the stored distance and traffic factor. Pending service requests are also retrieved from the database for priority-based processing.

This integration allows the implemented data structures and algorithms to operate on persistent University of Ghana campus data rather than relying solely on hard-coded test values.

9. Responsible Algorithm Selection

Algorithms were selected according to the nature of each campus-service problem, their correctness requirements, computational efficiency, and known limitations.

Algorithm	**Reason for Selection**	**Limitation / Condition**
BFS and DFS	Suitable for traversing the campus graph and determining connectivity or reachability between locations.	They identify reachable locations but do not independently determine the minimum-cost route in a weighted graph.
Dijkstra's Algorithm	Appropriate for determining shortest paths	Requires non-negative edge weights.

	between campus locations in a weighted network and uses a priority queue for efficient minimum-distance selection.	
Prim's and Kruskal's Algorithms	Suitable for constructing a Minimum Spanning Tree connecting campus locations with minimum total edge cost. Using both also provides	They solve network-wide minimum connectivity problems rather than point-to-point shortest-path problems.

	alternative approaches for validating MST results.	
Greedy Algorithm	Provides a simple and efficient approach for assigning each service request to the nearest available resource.	A locally optimal assignment may not produce the globally optimal overall allocation.
Dynamic Programming	Appropriate where an optimal subset of service requests or route options must be selected	Produces an optimal solution for the defined problem but requires greater time and memory as the problem size

	under a limited time or budget constraint.	and budget increase.

Dijkstra's Algorithm is appropriate for the campus routing problem because the implementation operates on weighted graph edges and maintains shortest-distance and predecessor information using a custom priority queue.

The Greedy approach was selected where rapid resource allocation is required; however, its implementation explicitly recognises that selecting the nearest resource for each request can lead to a suboptimal overall allocation. Dynamic Programming addresses problems requiring global optimisation by using 0/1 Knapsack tabulation and solution reconstruction to maximise priority or benefit within a specified constraint.

Therefore, algorithm selection in the project considers not only computational efficiency but also the assumptions, suitability, and limitations of each algorithm for the specific campus-service problem being addressed.

10. Individual Contribution Statement

Development of the **UG Campus Smart Service Operations Optimizer** was carried out collaboratively, with team members contributing to data structures, algorithms, database components, testing, system integration, user-interface functionality, and project documentation. Contributions were identified from the project implementation and Git version-control history.

Team Member	Main Contribution	Evidence
Emmanuel Grant Boamah	Created the initial repository and project structure; implemented the database connection, CSV data loader and database schema; corrected isolated campus locations and added the database reset utility; coordinated integration and merging of several feature branches; and contributed to presentation/demo preparation.	Commits 00b136e, 3346683, 2d883e5, a6f05a1, c3bb602; multiple merged PRs
Addo Michael Obiri	Implemented and tested the Dynamic Array, Stack and Circular Queue data structures, including fixes to their operations and associated unit tests.	Commits 75d753c, d30019a, a92c005, a783a7a, 5ad1a1d, 2856afb, 8d019bc, 442d178

Shahad Bawa Abdul-Raziq N-yaaba	Implemented the custom Linked List, Queue and Deque data structures.	Commits 31b9f1e, 66e8ca0, 0c52b66; PR #7 integration
Ethan Nii Arday Bill	Implemented the Greedy and Dynamic Programming algorithms and their tests; contributed estimated-time functionality required by Dynamic Programming; improved unit tests and integrated Greedy testing with the actual campus graph.	Commits 759c63c, a258e35, 319f012, 331a30f
Lord Dziedzom Yao Fianu	Implemented the initial versions of Prim's and Kruskal's Minimum Spanning Tree algorithms.	Commit f2cf647
Bridget Fafa Tossou	Worked on Feature 5, including Priority Queue-related functionality, and implemented subsequent requested changes and integration fixes.	Commits 1c0e604, 38a4d23; merge of feature/Priority_Queue_Bridget; PRs #13 and #18 from Bridget-papi
Christian Agyapong	Implemented major non-linear and indexing data structures, including the Binary Search Tree, Red-Black Tree, B-Tree, and hash-based Map/Set structures.	Commit 354b789
Mackey Kumi	Implemented BFS, DFS and Dijkstra's Algorithm; corrected and improved	Commits 5f4d936, db6c411; graph-algorithms integration

	Prim's and Kruskal's Algorithms; added supporting graph methods and tests.	
Gator Kimathi Jojo	Completed Feature 6 algorithm implementations and service-layer setup, including the final searching and sorting functionality used by the application.	Commit 214e7b9; merge 991dafc
Joseph Kwaku Wiafe	Developed and integrated major parts of the console user interface, including service-request operations, routing, searching, sorting, queue status, graph traversal and Minimum Spanning Tree functionality.	Commits 9925141, 4944a52, 100c8ca; merge df91c78; UI PR integration
Patience Owusu	Prepared and maintained the project technical documentation, including the problem definition, dataset and database documentation, architecture, data structures and algorithms, correctness and performance analysis, trace tables, proof sketches, counterexamples, references, appendices, contribution records, and final document organisation and formatting.	Project documentation prepared and maintained by the Documentation Team; repository evidence will be added after final approval.

Each team member is responsible for understanding and defending both his or her individual contribution and how that contribution integrates with the overall system.

11. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Oracle. (n.d.). *Java Platform, Standard Edition Documentation*. Oracle Corporation.

PostgreSQL Global Development Group. (n.d.). *PostgreSQL Documentation*.

JUnit Team. (n.d.). *JUnit 5 User Guide*.

Struct-Enqueue. (2026). *UG Campus Service Optimizer*. GitHub repository.

University of Ghana, Department of Computer Science. (2026). *DCIT 204/308 Joint Semester Project Documentation Template*.

12. Appendices

The appendices provide supporting evidence for the implementation, testing, database integration, and evaluation of the **UG Campus Smart Service Operations Optimizer**.

Appendix A: Project Source-Code Structure

Selected source-code files showing the organisation of the project into:

- Data Structures
- Algorithms
- Graph Algorithms
- Database Components
- Models
- Service Components
- User Interface
- Test Classes

Appendix B: Database and Dataset Evidence

Supporting database materials include:

- locations.csv
- routes.csv

- resources.csv
- requests.csv
- PostgreSQL database schema
- Database setup and CSV-loading implementation

Appendix C: Testing and Correctness Evidence

Relevant JUnit test results and selected test cases for:

- Data Structures
- BFS and DFS
- Dijkstra's Algorithm
- Prim's Algorithm
- Kruskal's Algorithm
- Greedy Algorithm
- Dynamic Programming

Appendix D: Performance Evidence

Performance experiment results, including:

- Input sizes
- Execution times
- Memory usage

- Expected complexities
- Observed performance
- Supporting tables or graphs where applicable

Appendix E: Version-Control Evidence

Selected GitHub evidence showing:

- Development branches
- Commits
- Pull requests
- Merged implementations
- Individual team-member contributions

Appendix F: Development Log

	Development Log					

27th July, 2026	8:00 PM	1	Initial team meeting to discuss the DSA semester project and understand the project requirements	1. Patience Owusu 2.Emmanuel Grant Boamah 3.Micheal Obiri 4.Abdul Raziq 5.Ethan Bill 6.Tossou Bridget Fafa 7.Christian Agyapong 8.Mackey Kumi	Team discussed the project requirements and deliverables.	Agreed to review the project instructions and meet again to assign roles.
28/07/2026	8:00 PM	2	Team planning and role assignment meeting	1.Boamah Emmanuel Grant 2. Patience Owusu 3. Mackey Kumi 4.Eshun Jeffrey 5.Tossou Bridget Fafa 6.Abdul-Raziq Nyaaba Shahad 7.Desmond Pinamang Aikins 8.Christian Agyapong 9.Kimathi Jojo Gator 10.Lord Fianu	The team assigned roles and responsibilities, discussed how the project would be organized, and agreed on the tasks each member would complete.	Roles were assigned based on members' strengths, and members agreed to begin working on their assigned tasks before the next meeting.
03/08/2026	8:00 PM	3	Work plan presentation and progress review	1. Boamah Emmanuel Grant 2. Owusu Patience 3. Tossou Bridget Fafa 4. Bill, Ethan Nii Addy 5.Amoah 6. Kimathi Jojo Gator 7.Addo Michael Obiri	The work plan was explained, and each member presented the work they had completed.	The team reviewed the work presented and agreed to continue with the assigned tasks according to the work plan.

05/08/2026	8:00 PM	4	Project work plan and feature implementation discussion	1. Boamah Emmanuel Grant 2.Abdul Raziq 3.Addo Michael Obiri 4.Joseph Kwaku Wiafe 5.Lord Fianu 6.Owusu Patience 7.Tossou Bridget Fafa	The team was taken through the project structure, GitHub repository setup, project features, and the data loader feature. The data requirements and the use of Java, CSV files, and a database were also discussed.	The team agreed to move from planning into implementation, join the GitHub organisation, work on assigned folders/features, and speed up development to meet the remaining project timeline.
26/08/2026	7:30 PM	5	Presentation and testing of completed work	1. Boamah Emmanuel Grant 2. Bill, Ethan Nii Arday 3. Lord Fianu 4. Owusu Patience 5. Tossou Bridget Fafa 6. Joseph Kwaku Wiafe	Team members presented the work they had completed and shared their screens to demonstrate and test the functionality of their work.	The team agreed to continue working on their assigned tasks and provide updates on their progress in subsequent meetings.

27/08/2026	12:30	6	The team reviewed the progress of the development work. Members discussed testing the implemented features, running the necessary commands, and checking that the system was working correctly. Preparations for the project presentation and demonstration were also discussed.	1.Boamah Emmanuel Grant 2.Abdul-Raziq Nyaaba Shahad B. 3.Tossou Bridget Fafa 4.Bill Ethan Nii Arday 5.Amoah 5.Joseph Kwaku Wiafe 6.Agyapong Christian 7Kimathi Jojo Gator 8.Owusu Patience	The team agreed to continue testing the work and ensure that the features were functioning properly. Members were also expected to prepare for the upcoming project presentation and demonstration.	Members should test their assigned features, confirm that their work is functioning correctly, and prepare their laptops for the project demonstration and presentation.

Appendix G: Individual Contribution Log

Member Name	Student ID	Role	Assigned Responsibilities	Work Completed	Evidence	Status	Remarks

Emmanuel Grant Boamah	22154941	Project/Integration Lead	Repository setup, database integration, data loading and feature integration	Created project structure; implemented DatabaseConnector, CSVLoader and SchemaSetup; fixed isolated campus locations; coordinated merges and demo preparation	00b136e, 3346683, 2d883e5, a6f05a1, c3bb602; merged PRs	Completed	Integrated several team features into dev
Gator Kimathi Jojo	22403299	Algorithm/Service Developer	Searching, sorting and service-layer implementation	Completed searching and sorting algorithms and related service setup	214e7b9, merge 991dafc	Completed	Feature 6 completed
Addo Michael Obiri	22182561	Data Structure Developer	Dynamic Array, Stack and Circular Queue	Implemented and tested Dynamic Array, Stack and Circular Queue; corrected related issues	75d753c, d30019a, a92c005, a783a7a, 5ad1a1d, 2856afb, 8d019bc, 442d178	Completed	Linear data structures tested
Shahad Bawa Abdul-Raziq N-yaaba	22399614	Data Structure Developer	Linked List, Queue and Deque	Implemented custom Linked List, Queue and Deque	31b9f1e, 66e8ca0, 0c52b66; PR #7	Completed	Integrated into graph branch

Ethan Nii Arday Bill	22410872	Algorithm Developer	Greedy and Dynamic Programming	Implemented Greedy and DP algorithms and tests; added estimated-time functionality; improved tests	759c63c, a258e35, 319f012, 331a30f	Completed	Greedy integrated with real campus graph
Lord Dziedzom Yao Fianu	22374358	Graph Algorithm Developer	Prim and Kruskal algorithms	Implemented initial Prim and Kruskal MST algorithms	f2cf647	Completed	Later refined during graph integration
Bridget Fafa Tossou	22406084	Data Structure Developer	Feature 5 and Priority Queue	Worked on Feature 5 and Priority Queue functionality; implemented requested changes	1c0e604, 38a4d23; feature/Priority_Queue_Bridget; PRs #13/#18	Completed	Priority Queue integrated into graph functionality
Christian Agyapong	22054189	Data Structure Developer	Tree and hash-based structures	Implemented BST, Red-Black Tree, B-Tree and hash-based Map/Set	354b789	Completed	Major indexing structures completed
Mackey Kumi	22025755	Graph Algorithm Developer	BFS, DFS, Dijkstra and MST improvements	Implemented BFS, DFS and Dijkstra; corrected Prim/Kruskal and added supporting tests	5f4d936, db6c411	Completed	Graph algorithms completed and tested
Joseph Kwaku Wiafe	22374055	UI/Integration Developer	Console interface and algorithm integration	Implemented console options for requests, routing, searching, sorting, graph traversal and MST operations	9925141, 4944a52, 100c8ca, merge df91c78	Completed	Integrated algorithms with console interface

Patience Owusu	22398703	Documentation Lead	Technical documentation and development records	Prepared technical report, architecture, dataset/database documentation, algorithm/data-structure documentation, testing/performance analysis, traces, proofs, counterexamples, contribution records and Excel development log	Final documentation and development log; Git commit/PR to be added after approval	Completed	Repository evidence to be updated after leader approval

PART II

TRACE TABLES

1. Binary Search Trace Table

Purpose: This trace table records the step-by-step execution of the Binary Search algorithm on a sorted dataset.

Sorted Array: [2, 4, 6, 8, 10, 12, 14]
Target: 10

Step	Left	Right	Mid	Value at Mid	Comparison	Action Taken
1	0	6	3	8	8 < 10	Search right half; set Left = 4
2	4	6	5	12	12 > 10	Search left half; set Right = 4
3	4	4	4	10	10 = 10	Target found at index 4

2. Insertion Sort Trace Table

Purpose: This trace table records each pass of the Insertion Sort algorithm, showing how the list changes after each insertion.

Initial Array: [5, 2, 6, 1, 3, 4]

Pass	Current Element	Array Before	Comparison	Array After
1	2	[5, 2, 6, 1, 3, 4]	5 > 2, so shift 5 right and insert 2 before it	[2, 5, 6, 1, 3, 4]
2	6	[2, 5, 6, 1, 3, 4]	5 < 6, so no shift is required	[2, 5, 6, 1, 3, 4]
3	1	[2, 5, 6, 1, 3, 4]	6 > 1, 5 > 1, 2 > 1; shift them right and insert 1	[1, 2, 5, 6, 3, 4]
4	3	[1, 2, 5, 6, 3, 4]	6 > 3 and 5 > 3; shift both right and insert 3 after 2	[1, 2, 3, 5, 6, 4]
5	4	[1, 2, 3, 5, 6, 4]	6 > 4 and 5 > 4; shift both right and insert 4 after 3	[1, 2, 3, 4, 5, 6]

3. Merge Sort Trace Table

Purpose: This trace table records how the Merge Sort algorithm divides the array into smaller subarrays and then merges them back into a sorted array.

Initial Array: [5, 2, 6, 1, 3, 4]

Step	Left Subarray	Right Subarray	Merged Result	Remarks
1	[5]	[2]	[2, 5]	Compare 5 and 2; place 2 first
2	[2, 5]	[6]	[2, 5, 6]	Merge produces sorted left half
3	[1]	[3]	[1, 3]	Compare 1 and 3
4	[1, 3]	[4]	[1, 3, 4]	Merge produces

				sorted right half
5	[2, 5, 6]	[1, 3, 4]	[1, 2, 3, 4, 5, 6]	Final merge of the two sorted halves

4. Dijkstra's Algorithm Trace

Purpose: This trace table records the step-by-step execution of Dijkstra's algorithm for finding the shortest path from a source location to other locations in the network.

Step	Current Node	Distance Update	Visited Nodes	Next Node
1	0	d(0)=0; d(1)=5; d(2)=2	{0}	2
2	2	d(3)=2+1=**3**	{0, 2}	3
3	3	d(4)=3+6=**9**	{0, 2, 3}	1
4	1	Existing d(3)=3 is shorter than 5+4=9; no update	{0, 2, 3, 1}	4
5	4	No shorter distance found	{0, 2, 3, 1, 4}	—

5. Kruskal's Algorithm Trace Table

Purpose: This trace table records the process of selecting edges during the construction of the Minimum Spanning Tree (MST).

Step	Edge Selected	Weight	Cycle Formed?	Action Taken
1	(2, 3)	1	No	Add edge to MST
2	(0, 2)	2	No	Add edge to MST
3	(1, 3)	4	No	Add edge to MST
4	(0, 1)	5	Yes	Reject edge
5	(3, 4)	6	No	Add edge to MST

6. Dynamic Programming Trace Table

Purpose: This trace table records how the Dynamic Programming table is filled step by step to obtain the optimal solution.

Step	**Current State**	**Decision Made**	**DP Table Update**	**Result**
1	No requests considered	Initialise base case	dp[0][w] = 0 for all budgets	Maximum priority = 0
2	Consider Request 1	Compare skipping the request with selecting it if its estimated time fits	dp[1][w] = max(dp[0][w], $value_1$ + dp[0][w-$cost_1$])	Best value for first request recorded

3	Consider Request 2	Select or skip according to which gives the greater total urgency	dp[2][w] = max(dp[1][w], $value_2$ + dp[1][w-$cost_2$])	Best value using Requests 1–2 recorded
4	Consider Request 3	Compare previous optimum with optimum obtained by including Request 3	dp[3][w] = max(dp[2][w], $value_3$ + dp[2][w-$cost_3$])	Best value using Requests 1–3 recorded
5	Continue for remaining requests	Repeat the take-or-skip decision for every request and	dp[i][w] = max(dp[i-1][w], $value_i$ + dp[i-1][w-$cost_i$])	Optimal values stored throughout the table

		available budget		
6	Final state dp[n][B]	Select the maximum achievable urgency within budget B	Read final DP value and backtrack through the table	Optimal subset of service requests obtained

PART III

PROOF SKETCHES

1. Binary Search Proof Sketch

Purpose: This proof sketch explains why the Binary Search algorithm correctly finds the target element in a sorted array or correctly reports that the element is not present.

Algorithm:
Binary Search

Claim:
Binary Search correctly finds the target element in a sorted array if the target is present, and correctly reports that the target is absent if it is not present.

Preconditions:

1. The input array must be sorted in ascending order.
2. The left and right boundaries must correctly represent the current search interval.
3. The target value must be comparable with the elements in the array.

Proof Idea:
At each step, Binary Search examines the middle element of the current search interval.

- If the middle element is equal to the target, the algorithm terminates successfully.

- If the target is smaller than the middle element, the target can only be in the left half because the array is sorted.
- If the target is larger than the middle element, the target can only be in the right half.

Therefore, each comparison safely eliminates half of the remaining search space without removing a possible location of the target. The search interval continues to decrease until the target is found or the interval becomes empty.

Conclusion:
Therefore, provided that the array is sorted, Binary Search correctly returns the position of the target when it exists and correctly reports that the target is absent when no valid search interval remains.

2. Insertion Sort Proof Sketch

Purpose: This proof sketch explains why the Insertion Sort algorithm correctly sorts a list into ascending order after all passes are complete.

Algorithm:
Insertion Sort

Claim:
Insertion Sort correctly sorts an array into ascending order after all passes are completed.

Preconditions:

1. The input is a valid array of comparable elements.
2. Elements can be compared using their ordering relationship.
3. The array may initially be in any order.

Proof Idea:
Insertion Sort maintains the property that the portion of the array already processed is sorted.

Initially, the first element alone is considered sorted. During each pass, the current element is compared with elements in the sorted portion. Any larger elements are shifted one position to the right until the correct position for the current element is found.

After inserting the current element, the processed portion remains sorted. This process continues until every element has been inserted into its correct position.

Conclusion:
Therefore, after all elements have been processed, the entire array is sorted in ascending order, proving that Insertion Sort correctly sorts the input array.

3. Dijkstra's Algorithm Proof Sketch

Purpose: This proof sketch explains why Dijkstra's Algorithm correctly finds the shortest path from a source node to all other reachable nodes in a graph with non-negative edge weights.

Algorithm:

Dijkstra's Algorithm

Claim:

Dijkstra's Algorithm correctly determines the shortest path from a given source location to every reachable location in the campus graph when all edge weights are non-negative.

Preconditions:

1. The campus network is represented as a weighted graph.
2. A valid source vertex is provided.
3. All route weights are non-negative.
4. Each edge correctly represents a route between two campus locations.

Proof Idea:

Initially, the source vertex is assigned distance 0, while all other vertices are assigned infinity. At each step, the algorithm selects the unvisited vertex with the smallest known distance. Since all edge weights are non-negative, no later path through an unvisited vertex can produce a shorter distance to the selected vertex. Its distance can therefore be considered final.

The algorithm then relaxes each neighbouring edge by checking whether travelling through the current vertex produces a shorter path. If so, the neighbour's distance and predecessor are updated. Repeating this process ensures that each reachable vertex eventually receives its minimum possible distance from the source.

Conclusion:

Therefore, when all route weights are non-negative, Dijkstra's Algorithm correctly computes the shortest distance and corresponding path from the source location to every reachable location in the campus network.

PART IV

COUNTEREXAMPLES

Purpose: This section records counterexamples that demonstrate situations where an algorithm fails or where its required assumptions are violated. These examples help explain the limitations of algorithms and the importance of satisfying their preconditions**.**

Counterexample 1

Greedy Algorithm Counterexample:

Purpose**:** This counterexample demonstrates a situation in which a greedy algorithm does not produce the optimal solution.

Algorithm: Greedy Resource Assignment Algorithm

Problem Description: The Greedy algorithm assigns each service request to the nearest available resource at the time the request is processed. Although this produces the best immediate choice, it may not produce the minimum overall assignment cost.

Counterexample Input: Assume there are two service requests, **R1** and **R2**, and two available resources, **A** and **B**.

Assignment	Distance
R1 → A	2
R1 → B	3
R2 → A	3
R2 → B	10

Expected Optimal Solution:

Assign **R1 → B** and **R2 → A**.

Total distance:

3 + 3 = 6

Greedy Algorithm Result:

The Greedy algorithm processes R1 first and chooses resource A because it is the nearest resource to R1.

- R1 → A = 2
- R2 → B = 10

Total distance:

2 + 10 = 12

Explanation:

The Greedy algorithm makes the locally optimal decision of assigning resource A to R1 because the distance is only 2. However, this leaves resource B for R2, resulting in a much larger total distance. Choosing the slightly more expensive assignment for R1 produces a better overall solution.

Conclusion:

This counterexample demonstrates that the Greedy resource-assignment algorithm can produce a locally optimal solution that is not globally optimal. Therefore, greedy allocation is efficient for rapid decision-making but does not always guarantee the minimum overall assignment cost.

Counterexample 2

Binary Search Precondition Counterexample

Purpose: This counterexample demonstrates what happens when Binary Search is applied to an unsorted array, violating one of its required preconditions.

Algorithm:
Binary Search

Problem Description:
Binary Search requires the input data to be sorted before the algorithm is applied. If the data is unsorted, the algorithm may eliminate the part of the array containing the target and therefore return an incorrect result.

Counterexample Input:

Array: **[10, 2, 8, 4, 6]**
Target: **2**

Expected Behaviour:
The algorithm should identify that the target value **2** is present in the array at index **1**.

Actual Behaviour:
Binary Search examines the middle value **8**. Since **2 < 8**, it searches only the left portion of the array. Depending on the subsequent comparisons, the target may be incorrectly reported as absent because the array is not sorted.

Explanation:
Binary Search assumes that all values smaller than the middle element occur on the left and all larger values occur on the right. This assumption is violated in an unsorted array. Therefore, discarding half of the search space is no longer logically valid.

Conclusion:
Binary Search should only be applied to sorted data. If the sorting precondition is violated, the algorithm cannot be guaranteed to return the correct result.

PART V

PERFORMANCE EXPERIMENT

Purpose: This section records the experimental performance of the implemented algorithms, including execution time and observations for different input sizes.

Algorithm	Input Size (n)	Test Case	Execution Time (ms)	Memory Usage (KB)	Expected Complexity	Observed Complexity	Remarks
Linear Search	10,000	Worst-case search	0.007002	0.000	O(n)	Approx. O(n)	Runtime generally increased as input size increased.
Binary Search	10,000	Sorted array search	0.000088	0.000	O(log n)	Approx. O(log n)	Very small increase in runtime

							despite large increase in input size.
Selection Sort	10,000	Random array	186.960200	0.000	$O(n^2)$	Approx. $O(n^2)$	Runtime increased substantially for larger arrays.
Insertion Sort	10,000	Random array	97.736900	0.000	$O(n^2)$	Approx. $O(n^2)$	Performance decreased significantly as array size increased.
Merge Sort	10,000	Random array	5.777600	695.359	O(n log n)	Approx. O(n log n)	Scaled efficiently but required additional

							memory for merging.
Quick Sort	10,000	Random array	1.126800	0.000	O(n log n) average	Approx. O(n log n)	Fastest sorting algorithm for the larger random datasets in this experiment.
BFS	1,000	Graph traversal	0.952800	229.570	O(V + E)	Approx. O(V + E)	Runtime and memory increased with graph size.
DFS	1,000	Graph traversal	0.474000	206.109	O(V + E)	Approx. O(V + E)	Efficient traversal with

							increasing graph size.
Dijkstra	1,000	Shortest-path computation	4.053300	540.945	O((V + E) log V)	Consistent with expected growth	Required more time and memory than basic graph traversal because shortest distances are maintained.
Prim	1,000	Minimum Spanning Tree	2.856600	312.383	O(E log V)	Consistent with expected growth	Efficient MST construction using minimum-weight

							candidate edges.
Kruskal	1,000	Minimum Spanning Tree	7.123700	263.398	O(E log E)	Consistent with expected growth	Edge sorting contributes significantly to execution time.
Greedy	200	Resource assignment	1.110600	1.008	O(R × M)	Approx. O(R × M), with timing variation	Generally fast, although individual timing measurements showed JVM/runtime variation.
Dynamic Programming	10,000	0/1 Knapsack, B = 200	18.922400	8096.570	O(nB)	Approx. O(nB)	Runtime and memory increased

							with input size; memory usage was highest because of DP tabulation.